

Pruebas de Primalidad

Prueba Básica ($O(\sqrt{n})$)

La idea para resolver la prueba de primalidad en este caso se basa en las siguientes observaciones:

- Todos los números primos a excepción del 2 son impares, por lo que todos los pares mayores que 2 se pueden excluir directamente. Lo mismo aplica para los múltiplos de 3, excepto el propio 3.
- Todo número compuesto n tiene al menos un factor primo menor o igual a $\sqrt{n}$ (si todos sus factores fueran mayores, su producto excedería n), por lo que no es necesario iterar hasta n , sino solo hasta $\sqrt{n}$.
- Todo entero se puede escribir en la forma $6k, 6k \pm 1, 6k + 2, 6k + 3$ o $6k + 4$. De estas, solo $6k \pm 1$ pueden ser primas (las demás son divisibles por 2 o 3), lo que permite saltar de 6 en 6 al iterar candidatos a divisor.

Esto da un algoritmo simple con complejidad temporal $O(\sqrt{n})$ y espacio constante:

```
1 bool isPrime(int n) {
2     if (n < 2) return false;
3     if (n == 2 || n == 3) return true;
4     if (n % 2 == 0 || n % 3 == 0) return false;
5     for (int i = 5; i * i <= n; i += 6) {
6         if (n % i == 0 || n % (i + 2) == 0) return false;
7     }
8     return true;
9 }
```

Este método es adecuado para valores de n de hasta 10^{12} aproximadamente en tiempo razonable, pero se vuelve impráctico para enteros de 64 bits, donde se necesita un enfoque más eficiente como Miller-Rabin.

Prueba de Miller-Rabin

El punto de partida es el **Pequeño Teorema de Fermat**: si n es primo y $\gcd(a, n) = 1$, entonces

$$a^{n-1} \equiv 1 \pmod{n}.$$

Como ya descartamos los pares, para un n impar el exponente $n - 1$ es par, así que se puede escribir como $n - 1 = d \cdot 2^s$, con d impar y $s \geq 1$. Factorizando $a^{n-1} - 1$ repetidamente como diferencia de cuadrados ($x^2 - 1 = (x + 1)(x - 1)$) se obtiene:

$$\begin{aligned} a^{2^s \cdot d} - 1 &\equiv 0 \pmod{n} \\ (a^{2^{s-1} \cdot d} + 1)(a^{2^{s-1} \cdot d} - 1) &\equiv 0 \pmod{n} \\ (a^{2^{s-1} \cdot d} + 1)(a^{2^{s-2} \cdot d} + 1)(a^{2^{s-2} \cdot d} - 1) &\equiv 0 \pmod{n} \\ &\vdots \\ (a^{2^{s-1} \cdot d} + 1)(a^{2^{s-2} \cdot d} + 1) \dots (a^d + 1)(a^d - 1) &\equiv 0 \pmod{n} \end{aligned}$$

Si n es primo, este producto solo puede ser $\equiv 0 \pmod{n}$ si al menos uno de sus factores lo es (ya que $\mathbb{Z}/n\mathbb{Z}$ es un cuerpo). Por lo tanto, para todo a con $2 \leq a \leq n-2$, si n es primo se debe cumplir una de las dos condiciones:

$$a^d \equiv 1 \pmod{n} \quad \text{o} \quad a^{2^r \cdot d} \equiv -1 \pmod{n} \quad \text{para algún } 0 \leq r \leq s-1.$$

Si *ninguna* de estas condiciones se cumple para una base a dada, entonces n es definitivamente compuesto, y decimos que a es un **testigo** de la composición de n . Si, en cambio, sí se cumple alguna condición pero n es compuesto, decimos que a es un **mentiroso fuerte** (*strong liar*) para n . Puede demostrarse que, para cualquier n compuesto e impar, a lo sumo $1/4$ de las bases posibles en $[2, n-2]$ son mentirosas. Esto significa que, probando k bases aleatorias independientes, la probabilidad de declarar erróneamente primo a un compuesto es a lo sumo 4^{-k} , valor que decrece exponencialmente y en la práctica se vuelve despreciable con pocas iteraciones.

Implementación Probabilística

Se utiliza exponenciación binaria rápida ($O(\log n)$ multiplicaciones) para evaluar cada base, y aritmética de 128 bits (`__uint128_t`) para evitar desbordamiento al multiplicar dos enteros de 64 bits.

Nota sobre la generación de bases aleatorias: usar `rand()` para elegir $a \in [2, n-2]$ es incorrecto para n de hasta 64 bits, ya que `rand()` normalmente solo produce valores hasta `RAND_MAX` (con frecuencia $2^{15} - 1$), muy por debajo del rango necesario. Esto hace que las bases probadas no cubran uniformemente $[2, n-2]$, debilitando la garantía probabilística del test. En su lugar se debe usar un generador de 64 bits como `std::mt19937_64` junto con `std::uniform_int_distribution`.

```

1 #include <random>
2 using u64 = uint64_t;
3 using u128 = __uint128_t;
4 u64 bp(u64 a, u64 b, u64 m){
5     u64 r=1; a%=m;
6     while(b){
7         if(b&1) r=(u128)r*a%m;
8         a=(u128)a*a%m;
9         b>>=1;
10    }
11    return r;
12 }
13 bool comp(u64 n, u64 a, u64 d, int s){
14     u64 x=bp(a,d,n);
15     if(x==1 || x==n-1) return false;
16     for(int r=1; r<s; r++){
17         x=(u128)x*x%n;
18         if(x==n-1) return false;
19     }
20     return
21     true;
22 }
23 bool mr(u64 n, int iter=20){
24     if(n<4) return n==2 || n==3;
25     if(n%2==0) return false;
26     int s=0;
27     u64 d=n-1;
28     while((d&1)==0){
29         d>>=1;

```

```

30     s++;
31 }
32 static mt19937_64 rng(random_device{}());
33 uniform_int_distribution<u64>dist(2,n-2);
34 for(int i=0;i<iter;i++){
35     u64 a=dist(rng);
36     if(comp(n,a,d,s))return false;
37 }
38 return true;
39 }

```

Con esta corrección, cada base a se elige uniformemente en $[2, n - 2]$, garantizando la cota de error 4^{-k} mencionada arriba. La complejidad total es $O(k \log n)$ multiplicaciones modulares, cada una en tiempo constante gracias a la aritmética de 128 bits.

Versión Determinista

Miller y luego Bach demostraron que, si se asume la Hipótesis de Riemann Generalizada (o, alternativamente, mediante verificación computacional exhaustiva para rangos acotados), es posible hacer el algoritmo determinista eligiendo un conjunto fijo de bases en lugar de bases aleatorias. En particular, se ha verificado que para evaluar la primalidad de cualquier número que quepa en un entero de 64 bits sin signo (`uint64_t`), basta con probar exhaustivamente las primeras 12 bases primas: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31 y 37. Este conjunto es determinista y correcto para todo $n < 3,317,044,064,679,887,385,961,981 \approx 3,3 \times 10^{24}$, rango que cubre por completo los 64 bits.

```

1 bool mrdet(u64 n){
2     if(n<2)return false;
3     if(n==2||n==3)return true;
4     if(n%2==0)return false;
5     int s=0;
6     u64 d=n-1;
7     while((d&1)==0){
8         d>>=1;
9         s++;
10    }
11    for(int a:{2,3,5,7,11,13,17,19,23,29,31,37}){
12        if(n==a)return true;
13        if(comp(n,a,d,s))return false;
14    }
15    return true;
16 }

```

La comprobación `if (n == a) return true;` es necesaria porque, si n coincide exactamente con una de las bases de prueba, `check_composite` intentaría evaluar a módulo n (dando 0), lo que produciría un resultado incorrecto; en ese caso ya sabemos que n es primo por definición de la lista de bases. Esta versión determinista tiene complejidad $O(12 \log n)$, es decir, $O(\log n)$ con una constante fija, y es la opción recomendada cuando se necesita certeza absoluta (no solo probabilística) para enteros de 64 bits.